

1.

```
In [12]: import numpy as np
import scipy.stats as statistics
import matplotlib.pyplot as plot

total_reservations = 43
plane_seats = 40

probability_values = np.array([i / 100 for i in range(70, 91)])

probability_three_empty = []
probability_one_or_more_empty = []

for probability in probability_values:
    binomial_distribution = statistics.binom(total_reservations, probability)

    probability_37 = binomial_distribution.pmf(37)
    probability_three_empty.append(probability_37)

    probability_at_most_40 = binomial_distribution.cdf(40)
    probability_one_or_more_empty.append(probability_at_most_40)

best_probability_a = probability_values[np.argmax(probability_three_empty)]

probability_b_less_95 = probability_values[
    np.array(probability_one_or_more_empty) < 0.95
]

# Print results
print(
    f"(a): \n"
    + f"Probability of missing 3 seats for each p: {[round(float(value), 3)
    + f"Maximum probability of exactly 3 empty seats occurs at probability =
    )
print(
    f"(b): \n"
    + f"Probabilities of having at least one empty seat: {[f'{probability_va
    + f"Probability values where probability of at least one empty seat < 0.
    )
```

(a):

Probability of missing 3 seats for each p: [0.008, 0.011, 0.015, 0.021, 0.027, 0.035, 0.045, 0.057, 0.07, 0.085, 0.101, 0.118, 0.134, 0.149, 0.161, 0.17, 0.173, 0.17, 0.161, 0.145, 0.124]

Maximum probability of exactly 3 empty seats occurs at probability = 0.860

(b):

Probabilities of having at least one empty seat: ['0.7: 1.0', '0.71: 1.0', '0.72: 1.0', '0.73: 1.0', '0.74: 1.0', '0.75: 1.0', '0.76: 0.999', '0.77: 0.999', '0.78: 0.998', '0.79: 0.997', '0.8: 0.995', '0.81: 0.993', '0.82: 0.989', '0.83: 0.984', '0.84: 0.977', '0.85: 0.966', '0.86: 0.951', '0.87: 0.931', '0.88: 0.903', '0.89: 0.866', '0.9: 0.818']

Probability values where probability of at least one empty seat < 0.95: [0.87 0.88 0.89 0.9]

2.

```
In [13]: # Hypergeometric

# a.  $P(X = 4) = (C(7,4) * C(3,0)) / C(10,4)$ 
# b.  $P(X \geq 2) = 1 - P(X = 0) - P(X = 1)$ 

from math import comb

def hypergeom_prob(success, total_success, sample, total):
    return (comb(total_success, success) * comb(total - total_success, sample - success)) / comb(total, sample)

p_all_fire = hypergeom_prob(4, 7, 4, 10)
p_at_most_2_fail = 1 - hypergeom_prob(0, 7, 4, 10) - hypergeom_prob(1, 7, 4, 10)

p_all_fire, p_at_most_2_fail
```

Out[13]: (0.16666666666666666, 0.9666666666666667)

3.

```
In [15]: # Poisson

#  $P(X < 7) = \sum P(X = k)$  for  $k = 0$  to  $6$ , where  $P(X = k) = (e^{-12} * 12^k) / k!$ 

from math import exp, factorial

def poisson_prob(k, lambd):
    return (exp(-lambd) * lambd**k) / factorial(k)

p_fewer_than_7 = sum(poisson_prob(k, 12) for k in range(7))

p_fewer_than_7
```

Out[15]: 0.045822306888651125

4.

```
In [16]: # Binomial

# a.  $P(X = 0) = (C(5,0) * 0.35^0 * 0.65^5)$ 
# b.  $P(X < 2) = P(X = 0) + P(X = 1)$ 
# c.  $P(X > 3) = P(X = 4) + P(X = 5)$ 

from math import comb

def binomial_prob(success, trials, prob):
    return comb(trials, success) * (prob**success) * ((1-prob)**(trials-success))

p_none = binomial_prob(0, 5, 0.35)
p_fewer_than_2 = p_none + binomial_prob(1, 5, 0.35)
p_more_than_3 = binomial_prob(4, 5, 0.35) + binomial_prob(5, 5, 0.35)

p_none, p_fewer_than_2, p_more_than_3
```

```
Out[16]: (0.11602906250000002, 0.42841500000000005, 0.05402249999999999)
```

5.

```
In [17]: # Poisson

# a.  $P(X = 4) = (e^{(-6)} * 6^4) / 4!$ 
# b.  $P(X \geq 4) = 1 - \sum P(X = k) \text{ for } k = 0 \text{ to } 3$ 
# c.  $P(X = 75) = (e^{(-72)} * 72^{75}) / 75!$ 

from math import exp, factorial

def poisson_prob(k, lambd):
    return (exp(-lambd) * lambd**k) / factorial(k)

p_exactly_4 = poisson_prob(4, 6)
p_at_least_4 = 1 - sum(poisson_prob(k, 6) for k in range(4))
p_exactly_75 = poisson_prob(75, 72)

p_exactly_4, p_at_least_4, p_exactly_75
```

```
Out[17]: (0.13385261753998337, 0.8487961172233521, 0.04326361560291038)
```

6.

```
In [18]: # Binomial

# a.  $P(3 \leq X \leq 6) = P(X = 3) + P(X = 4) + P(X = 5) + P(X = 6)$ 
# b.  $P(X < 4) = P(X = 0) + P(X = 1) + P(X = 2) + P(X = 3)$ 
# c.  $P(X > 5) = P(X = 6) + P(X = 7) + \dots + P(X = 15)$ 
```

```

p_3_to_6 = sum(binomial_prob(k, 15, 0.25) for k in range(3, 7))
p_fewer_than_4 = sum(binomial_prob(k, 15, 0.25) for k in range(4))
p_more_than_5 = sum(binomial_prob(k, 15, 0.25) for k in range(6, 16))

p_3_to_6, p_fewer_than_4, p_more_than_5

```

Out[18]: (0.707291878759861, 0.46128687635064125, 0.148368077352643)

7.

```

In [19]: # Negative Binomial Distribution

# a.  $P(X = 3) = (1 - p)^{(3-1)} * p$ 
# b.  $P(X = 7) = (1 - p)^{(7-3)} * p$ 
# c. Mean =  $r / p$ , Variance =  $r * (1 - p) / p^2$ 

p = 0.2
r = 3

# a. Probability that the first strike comes on the third well
p_first_strike_3 = (1 - p)**(3-1) * p

# b. Probability that the third strike comes on the seventh well
p_third_strike_7 = (1 - p)**(7-3) * p

# c. Mean and variance for 3 strikes
mean = r / p
variance = r * (1 - p) / p**2

p_first_strike_3, p_third_strike_7, mean, variance

```

Out[19]: (0.12800000000000003, 0.08192000000000002, 15.0, 60.0)